03_dags_basics

May 4, 2026

1 Directed Acyclic Graphs (DAGs) for Causal Inference

1.1 Learning Objectives

By the end of this notebook, you will be able to: - Understand the basic structure and rules of DAGs - Construct DAGs to represent causal relationships - Identify different types of paths: chains, forks, and colliders - Recognize confounders, mediators, and colliders in DAGs - Use DAGs to reason about which variables to control for - Visualize and analyze DAGs using Python

---

1.2 Overview

Directed Acyclic Graphs (DAGs) provide a visual and mathematical framework for representing causal relationships. They complement the potential outcomes framework we learned earlier.

Key idea: Draw arrows from causes to effects. The graph structure tells us: - Which variables are causes and which are effects - How information flows through the system - Which variables we need to control for to estimate causal effects

1.2.1 Quick Theory Recap

Components: - **Nodes** (circles/boxes): Variables in the system - **Directed edges** (arrows): Direct causal effects - **No cycles**: Can't have X → Y → X (that's why it's "acyclic")

The DAG encodes our causal assumptions about how the world works. It's not learned from data—it's based on domain knowledge, theory, and logic.

Let's dive in with code and examples!

1.3 Setup

[1]:
```
# Install dependencies if needed
# Uncomment the line below if this is your first time running the notebook
# !pip install -r ../requirements.txt
```

[2]:
```
# Import libraries
import numpy as np
import pandas as pd
```

```python
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
from typing import List, Tuple, Set

# Set style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (8, 5)
np.random.seed(42)

print("Libraries imported successfully!")
print(f"NetworkX version: {nx.__version__}")
```

```
Libraries imported successfully!
NetworkX version: 3.5
```

1.4 Part 1: Creating and Visualizing DAGs

Let's start by creating simple DAGs and visualizing them.

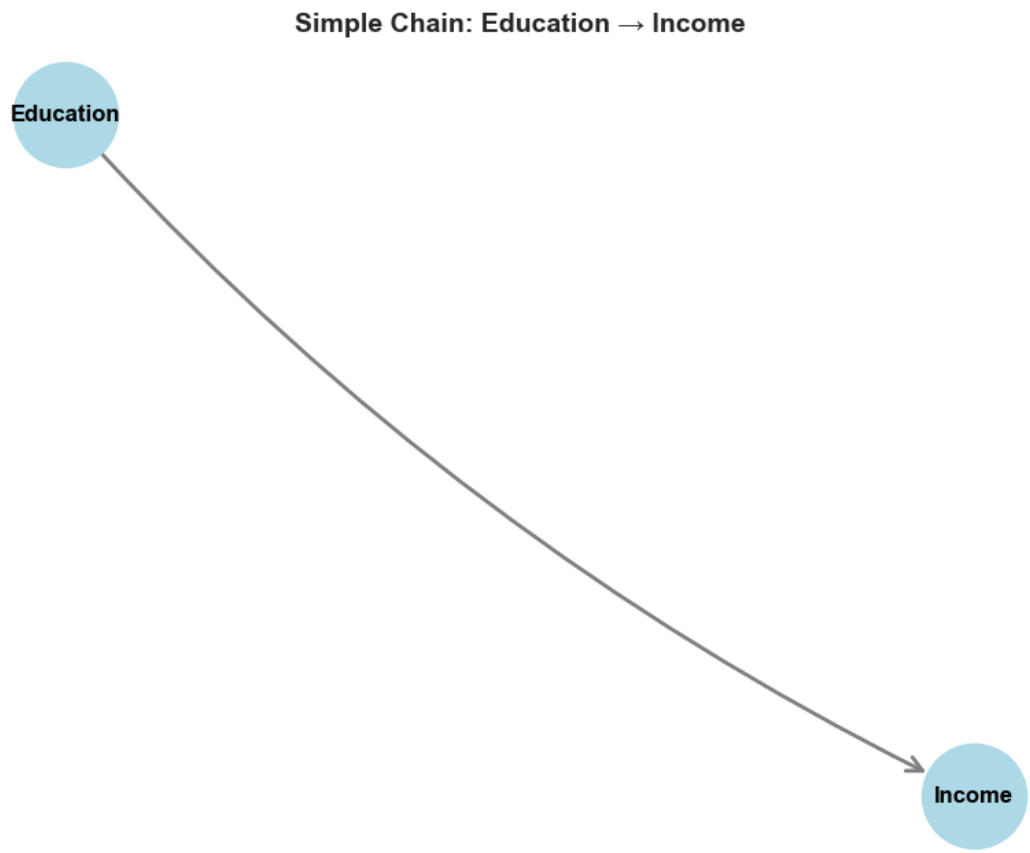
1.4.1 Example 1: Simple Chain - Education → Income

The simplest DAG: one variable causes another.

[3]:
```python
def draw_dag(edges: List[Tuple[str, str]], title: str = "DAG", node_size: int =␣
↪3000,
             font_size: int = 12, figsize: Tuple[int, int] = (8, 6)):
    """
    Draw a directed acyclic graph.

    Parameters
    ----------
    edges : List[Tuple[str, str]]
        List of (source, target) tuples representing directed edges
    title : str
        Title for the plot
    node_size : int
        Size of nodes in the graph
    font_size : int
        Font size for node labels
    figsize : Tuple[int, int]
        Figure size (width, height)
    """
    # Create directed graph
    G = nx.DiGraph()
    G.add_edges_from(edges)
```

```python
    # Create figure
    plt.figure(figsize=figsize)

    # Use hierarchical layout for better visualization
    pos = nx.spring_layout(G, k=2, iterations=50, seed=42)

    # Draw the graph
    nx.draw(G, pos,
            with_labels=True,
            node_color='lightblue',
            node_size=node_size,
            font_size=font_size,
            font_weight='bold',
            arrows=True,
            arrowsize=20,
            arrowstyle='->',
            edge_color='gray',
            width=2,
            connectionstyle='arc3,rad=0.1')

    plt.title(title, fontsize=14, fontweight='bold')
    plt.axis('off')
    plt.tight_layout()
    plt.show()

    return G

# Example 1: Simple causal chain
edges_simple = [('Education', 'Income')]
G_simple = draw_dag(edges_simple, title="Simple Chain: Education → Income")

print("\nInterpretation: Education has a direct causal effect on Income.")
print("The arrow direction shows causation flows from Education to Income.")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_7987/3657559720.py:45
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
Interpretation: Education has a direct causal effect on Income.
The arrow direction shows causation flows from Education to Income.
```

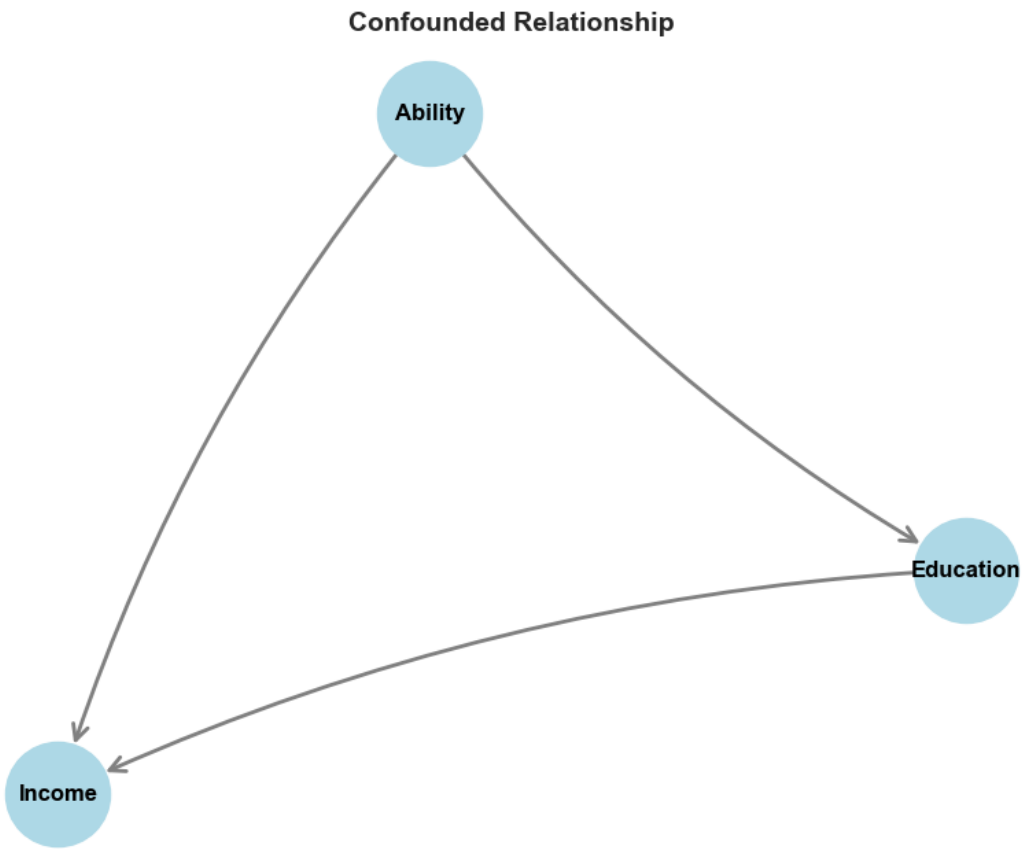
1.4.2 Example 2: Confounding - The Classic Problem

Now let's add a confounder: a variable that affects both treatment and outcome.

[4]:
```
# Example 2: Confounding
edges_confounded = [
    ('Ability', 'Education'),
    ('Ability', 'Income'),
    ('Education', 'Income')
]

G_confounded = draw_dag(edges_confounded, title="Confounded Relationship")

print("\nInterpretation:")
print("  • Ability affects both Education and Income (CONFOUNDER)")
print("  • Education also affects Income (direct effect)")
```

```
print("  • If we don't control for Ability, we'll get biased estimates of␣
↪Education → Income")
print("  • This creates a 'backdoor path': Education ← Ability → Income")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_7987/3657559720.py:45
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
Interpretation:
  • Ability affects both Education and Income (CONFOUNDER)
  • Education also affects Income (direct effect)
  • If we don't control for Ability, we'll get biased estimates of Education →
Income
  • This creates a 'backdoor path': Education ← Ability → Income
```

1.5 Part 2: Three Fundamental Structures

There are three basic building blocks in DAGs. Understanding these is key to causal reasoning.

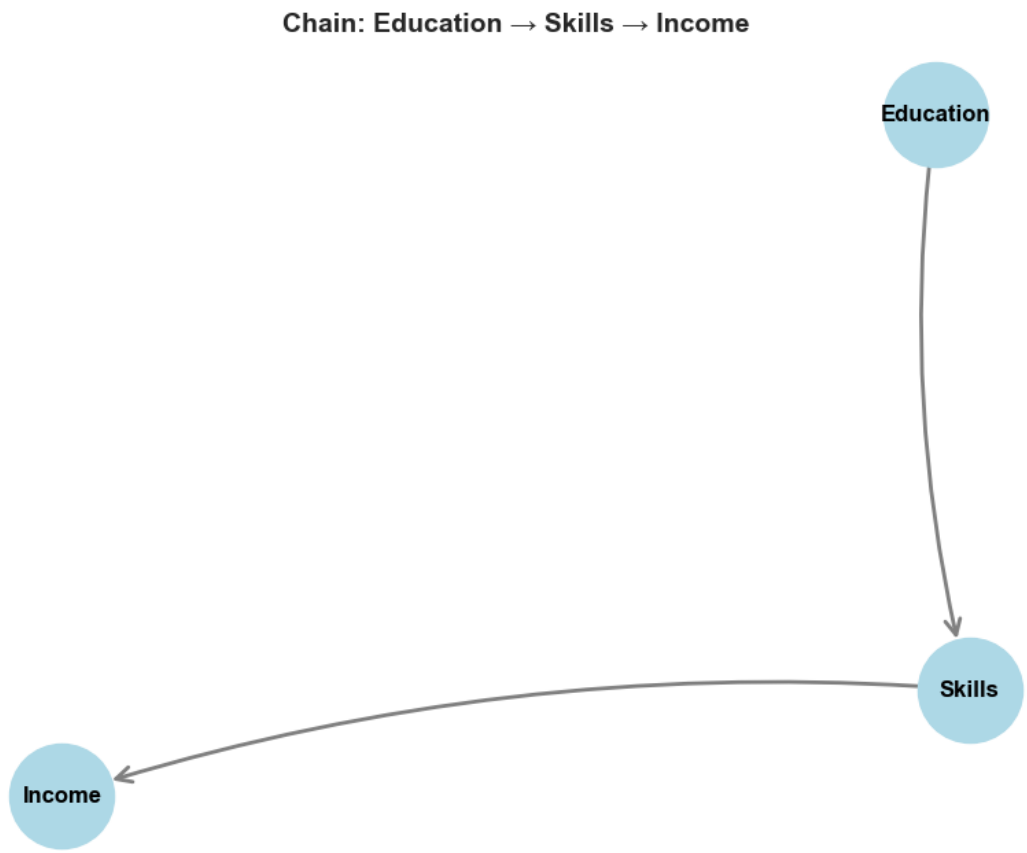
1.5.1 Structure 1: Chain (Mediation)

X → M → Y

M is a **mediator**: X affects Y through M.

[5]:
```
# Chain structure
edges_chain = [
    ('Education', 'Skills'),
    ('Skills', 'Income')
]

G_chain = draw_dag(edges_chain, title="Chain: Education → Skills → Income")

print("\nChain Structure (Mediation):")
print("  • Education affects Skills")
print("  • Skills affect Income")
print("  • Skills MEDIATES the effect of Education on Income")
print("\n  What happens if we control for Skills?")
print("  → We BLOCK the path from Education to Income")
print("  → Can't estimate total effect of Education (only direct effect, if␣
↪any)")
print("\n  Key insight: DON'T control for mediators if you want the total␣
↪effect!")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_7987/3657559720.py:45
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
Chain Structure (Mediation):
  • Education affects Skills
  • Skills affect Income
  • Skills MEDIATES the effect of Education on Income

  What happens if we control for Skills?
  → We BLOCK the path from Education to Income
  → Can't estimate total effect of Education (only direct effect, if any)

  Key insight: DON'T control for mediators if you want the total effect!
```

1.5.2 Structure 2: Fork (Common Cause / Confounding)

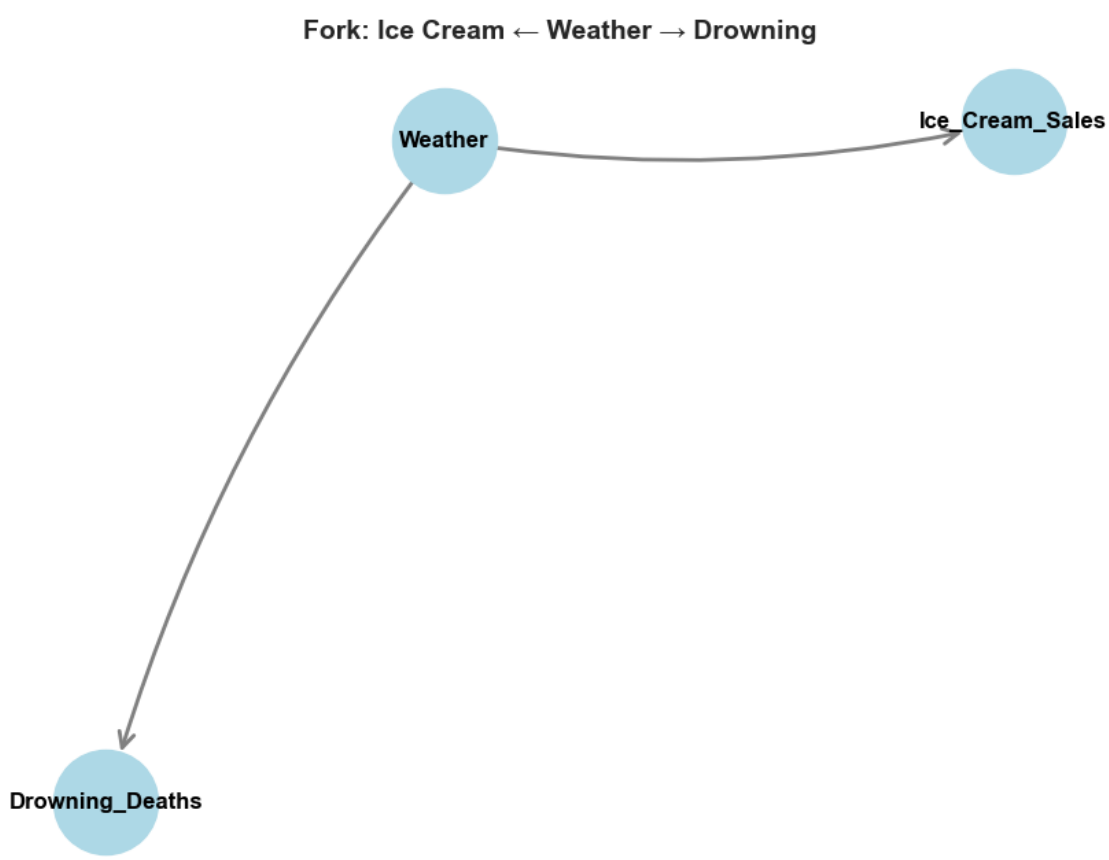
X ← Z → Y

Z is a **confounder**: a common cause of both X and Y.

[6]:
```
# Fork structure
edges_fork = [
    ('Weather', 'Ice_Cream_Sales'),
```

```
    ('Weather', 'Drowning_Deaths')
]

G_fork = draw_dag(edges_fork, title="Fork: Ice Cream ← Weather → Drowning")

print("\nFork Structure (Common Cause):")
print("  • Weather affects both Ice Cream Sales and Drowning Deaths")
print("  • Ice Cream and Drowning are CORRELATED but not causally related")
print("  • Weather is a CONFOUNDER")
print("\n  What happens if we control for Weather?")
print("  → We BLOCK the spurious association")
print("  → Ice Cream and Drowning become independent")
print("\n  Key insight: DO control for confounders to eliminate bias!")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_7987/3657559720.py:45
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```



```
Fork Structure (Common Cause):
  • Weather affects both Ice Cream Sales and Drowning Deaths
  • Ice Cream and Drowning are CORRELATED but not causally related
  • Weather is a CONFOUNDER

  What happens if we control for Weather?
  → We BLOCK the spurious association
  → Ice Cream and Drowning become independent

  Key insight: DO control for confounders to eliminate bias!
```

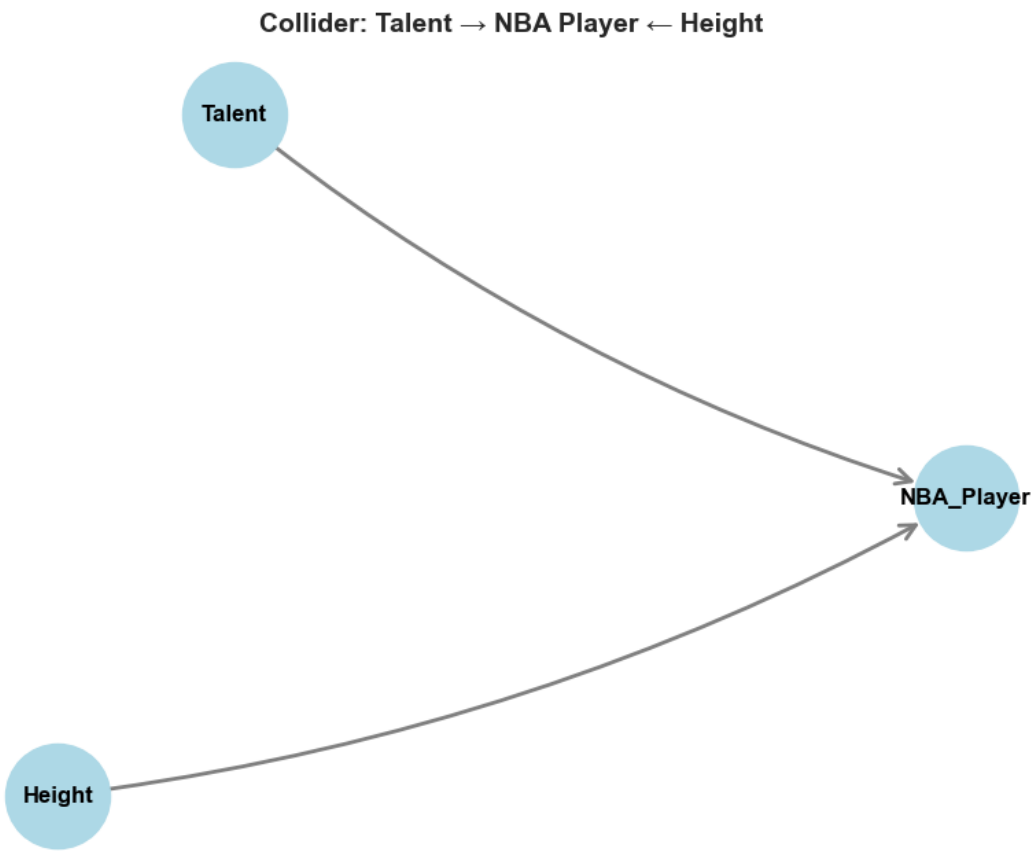
1.5.3 Structure 3: Collider (Selection Bias)

X → C ← Y

C is a **collider**: a common effect of both X and Y.

[7]:
```
# Collider structure
edges_collider = [
    ('Talent', 'NBA_Player'),
    ('Height', 'NBA_Player')
]

G_collider = draw_dag(edges_collider, title="Collider: Talent → NBA Player ←␣
↪Height")

print("\nCollider Structure:")
print("  • Talent and Height both affect becoming an NBA Player")
print("  • NBA_Player is a COLLIDER")
print("  • Talent and Height are INDEPENDENT in the general population")
print("\n  What happens if we control for (condition on) NBA_Player?")
print("  → We OPEN a spurious association (selection bias)")
print("  → Among NBA players, Talent and Height become NEGATIVELY correlated")
print("  → (Short NBA players must be exceptionally talented to compensate)")
print("\n  Key insight: DON'T control for colliders - creates bias!")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_7987/3657559720.py:45
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```

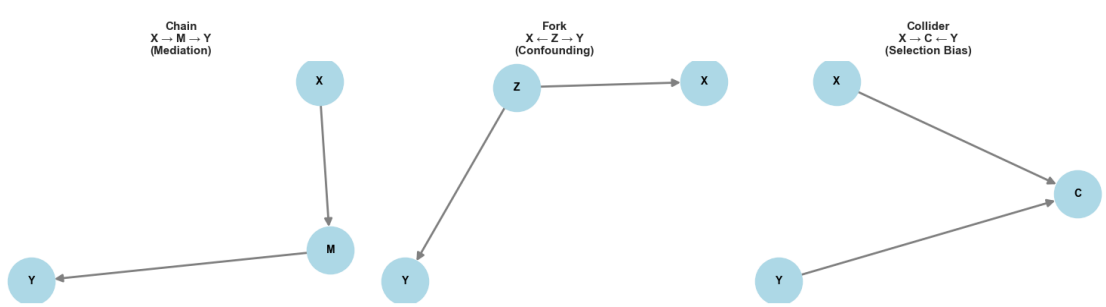



```
Collider Structure:
  • Talent and Height both affect becoming an NBA Player
  • NBA_Player is a COLLIDER
  • Talent and Height are INDEPENDENT in the general population

  What happens if we control for (condition on) NBA_Player?
  → We OPEN a spurious association (selection bias)
  → Among NBA players, Talent and Height become NEGATIVELY correlated
  → (Short NBA players must be exceptionally talented to compensate)

  Key insight: DON'T control for colliders - creates bias!
```

1.5.4 Summary of Three Structures

[8]:
```
# Visualize all three together
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

structures = [
    ([('X', 'M'), ('M', 'Y')], 'Chain\nX → M → Y\n(Mediation)'),
```

```
    ([('Z', 'X'), ('Z', 'Y')], 'Fork\nX ← Z → Y\n(Confounding)'),
    ([('X', 'C'), ('Y', 'C')], 'Collider\nX → C ← Y\n(Selection Bias)')
]

for idx, (edges, title) in enumerate(structures):
    G = nx.DiGraph()
    G.add_edges_from(edges)

    plt.sca(axes[idx])
    pos = nx.spring_layout(G, k=2, iterations=50, seed=42)
    nx.draw(G, pos,
            with_labels=True,
            node_color='lightblue',
            node_size=2000,
            font_size=10,
            font_weight='bold',
            arrows=True,
            arrowsize=15,
            edge_color='gray',
            width=2,
            ax=axes[idx])
    axes[idx].set_title(title, fontsize=11, fontweight='bold')
    axes[idx].axis('off')

plt.tight_layout()
plt.show()

print("\n Control for:")
print("    Forks (confounders) - to block backdoor paths")
print("    Chains (mediators) - would block the causal effect")
print("    Colliders - would create spurious associations")
```




```
 Control for:
   Forks (confounders) - to block backdoor paths
   Chains (mediators) - would block the causal effect
```

```
Colliders - would create spurious associations
```

1.6 Part 3: Paths and d-separation

Paths connect variables in a DAG. Understanding paths is crucial for identifying confounding.

Types of paths: - **Directed path**: Follows arrow directions (causal path) - **Backdoor path**: Goes against at least one arrow (confounding path)

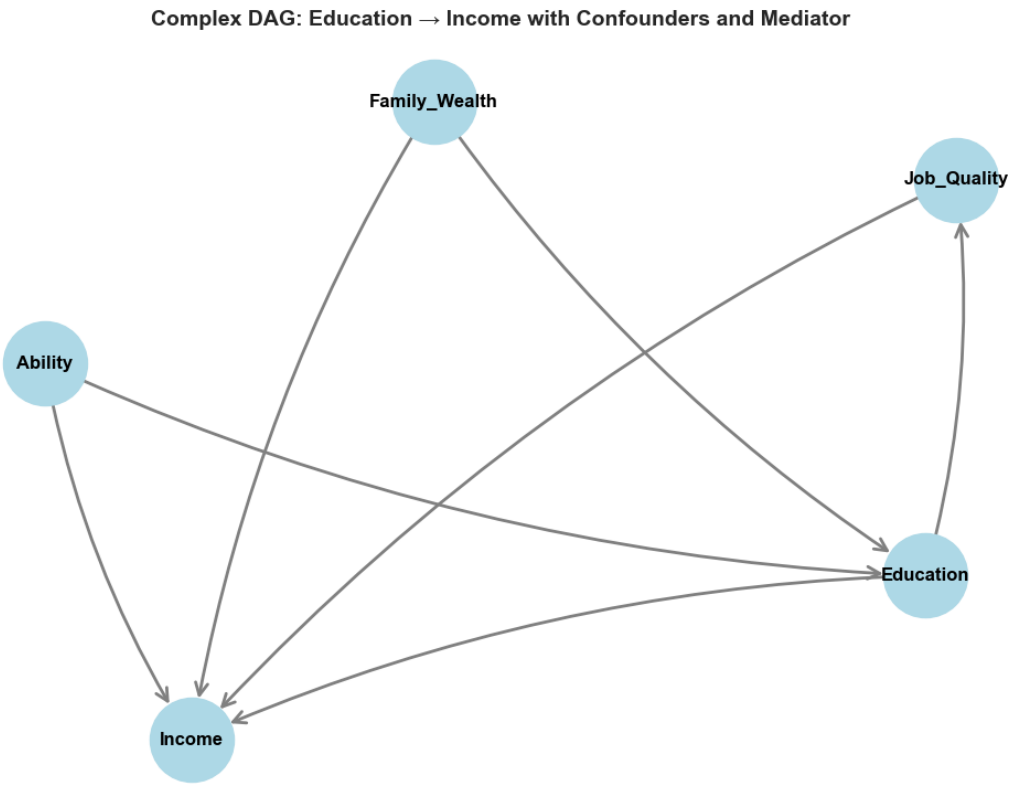
d-separation: Two variables are d-separated if all paths between them are blocked.

1.6.1 Example: Education → Income with Multiple Confounders

[9]:
```
# Complex DAG with multiple paths
edges_complex = [
    # Confounders
    ('Family_Wealth', 'Education'),
    ('Family_Wealth', 'Income'),
    ('Ability', 'Education'),
    ('Ability', 'Income'),
    # Direct effect
    ('Education', 'Income'),
    # Mediator
    ('Education', 'Job_Quality'),
    ('Job_Quality', 'Income')
]

G_complex = draw_dag(edges_complex,
                     title="Complex DAG: Education → Income with Confounders␣
 ↪and Mediator",
                     figsize=(10, 7))

print("\nPaths from Education to Income:")
print("\n1. DIRECT PATH (causal):")
print("   Education → Income")

print("\n2. MEDIATED PATH (causal):")
print("   Education → Job_Quality → Income")

print("\n3. BACKDOOR PATH (confounding) via Family_Wealth:")
print("   Education ← Family_Wealth → Income")

print("\n4. BACKDOOR PATH (confounding) via Ability:")
print("   Education ← Ability → Income")

print("\n  To estimate the causal effect of Education on Income:")
print("   • MUST control for: Family_Wealth and Ability (confounders)")
```

```
print("    • MUST NOT control for: Job_Quality (mediator)")
print("    • Controlling for confounders blocks backdoor paths")
print("    • Leaving mediator alone preserves total causal effect")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_7987/3657559720.py:45
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
Paths from Education to Income:

1. DIRECT PATH (causal):
   Education → Income

2. MEDIATED PATH (causal):
   Education → Job_Quality → Income

3. BACKDOOR PATH (confounding) via Family_Wealth:
   Education ← Family_Wealth → Income

4. BACKDOOR PATH (confounding) via Ability:
```

```
  Education ← Ability → Income

 To estimate the causal effect of Education on Income:
   • MUST control for: Family_Wealth and Ability (confounders)
   • MUST NOT control for: Job_Quality (mediator)
   • Controlling for confounders blocks backdoor paths
   • Leaving mediator alone preserves total causal effect
```

1.6.2 Finding Paths Programmatically

[10]:
```python
def find_all_paths(G: nx.DiGraph, source: str, target: str,
                   undirected: bool = True) -> List[List[str]]:
    """
    Find all paths between source and target in a DAG.

    Parameters
    ----------
    G : nx.DiGraph
        The directed graph
    source : str
        Source node
    target : str
        Target node
    undirected : bool
        If True, treat graph as undirected (to find all paths including␣
 ↪backdoor)

    Returns
    -------
    List[List[str]]
        List of paths (each path is a list of nodes)
    """
    if undirected:
        # Convert to undirected to find all paths
        G_undirected = G.to_undirected()
        paths = list(nx.all_simple_paths(G_undirected, source, target))
    else:
        # Only directed paths
        paths = list(nx.all_simple_paths(G, source, target))

    return paths

# Find all paths from Education to Income
all_paths = find_all_paths(G_complex, 'Education', 'Income', undirected=True)

print("All paths from Education to Income:\n")
for i, path in enumerate(all_paths, 1):
```

```
    path_str = ' → '.join(path)
    print(f"{i}. {path_str}")

print(f"\nTotal number of paths: {len(all_paths)}")
```

```
All paths from Education to Income:

1. Education → Family_Wealth → Income
2. Education → Income
3. Education → Job_Quality → Income
4. Education → Ability → Income

Total number of paths: 4
```

[11]:
```
# Find all paths from Education to Income
# set undirected=False to only get directed paths
all_paths = find_all_paths(G_complex, 'Education', 'Income', undirected=False)

print("All paths from Education to Income:\n")
for i, path in enumerate(all_paths, 1):
    path_str = ' → '.join(path)
    print(f"{i}. {path_str}")

print(f"\nTotal number of paths: {len(all_paths)}")
```

```
All paths from Education to Income:

1. Education → Income
2. Education → Job_Quality → Income

Total number of paths: 2
```

1.7 Part 4: Simulating Data from a DAG

Let's generate data that follows the causal structure specified by a DAG. This helps us verify that our understanding is correct.

1.7.1 Example: Simulating the Confounded Education-Income Relationship

[12]:
```
# Simulate data following the DAG structure
def simulate_education_income(n: int = 1000) -> pd.DataFrame:
    """
    Simulate data following this DAG:
    Ability → Education → Income
```

```python
    Ability is a confounder.
    """
    np.random.seed(42)

    # Ability (confounder)
    ability = np.random.normal(0, 1, n)

    # Education depends on ability
    education = 12 + 2 * ability + np.random.normal(0, 1, n)

    # Income depends on BOTH ability and education
    income = 30 + 3 * ability + 2 * education + np.random.normal(0, 3, n)

    return pd.DataFrame({
        'ability': ability,
        'education': education,
        'income': income
    })

# Generate data
df = simulate_education_income(n=1000)

print("Data generated from DAG:")
print("\nStructural equations:")
print("  Ability ~ N(0, 1)")
print("  Education = 12 + 2*Ability +  ")
print("  Income = 30 + 3*Ability + 2*Education +  ")
print("\nTrue causal effect of Education on Income: 2")
print("\nFirst 5 rows:")
print(df.head())
```

```
Data generated from DAG:

Structural equations:
  Ability ~ N(0, 1)
  Education = 12 + 2*Ability +
  Income = 30 + 3*Ability + 2*Education +

True causal effect of Education on Income: 2

First 5 rows:
    ability  education     income
0  0.496714  14.392784  58.250175
1 -0.138264  12.648105  54.447861
2  0.647689  13.355007  56.275821
3  1.523030  14.399123  62.443451
4 -0.234153  12.229917  48.076529
```

1.7.2 Naive vs Adjusted Estimates

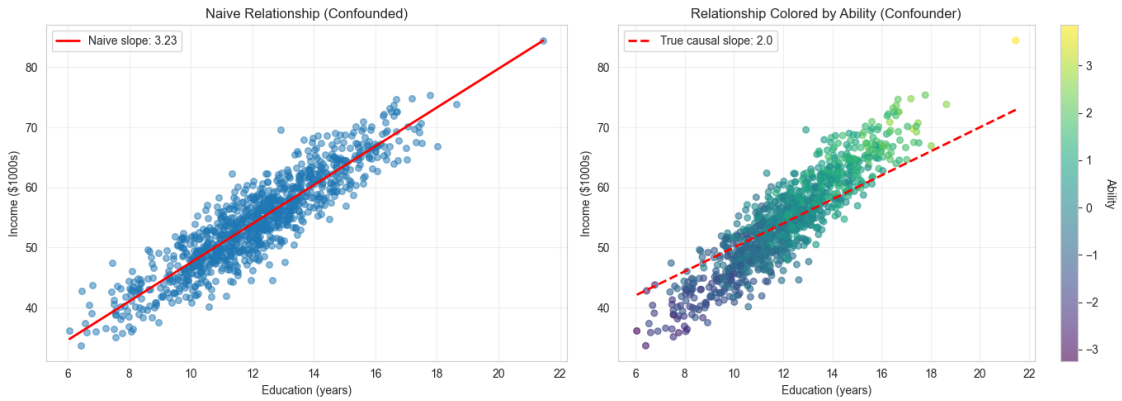
[13]:
```
from sklearn.linear_model import LinearRegression

# Naive estimate (without controlling for confounder)
# In this example, confounder is Ability, which affects both Education and␣
↪Income
# Recall: Education also influences Income,
# but if we don't control for Ability, we get a biased estimate of the effect␣
↪of Education on Income
X_naive = df[['education']].values
y = df['income'].values

model_naive = LinearRegression()
model_naive.fit(X_naive, y)
coef_naive = model_naive.coef_[0]

# Adjusted estimate (controlling for ability)
X_adjusted = df[['education', 'ability']].values

model_adjusted = LinearRegression()
model_adjusted.fit(X_adjusted, y)
coef_adjusted = model_adjusted.coef_[0]  # Coefficient for education

print("Regression Results:\n")
print(f"TRUE causal effect of Education on Income: 2.0")
print(f"\nNaive estimate (biased):    {coef_naive:.3f}")
print(f"Adjusted estimate (correct): {coef_adjusted:.3f}")
print(f"\nBias from not controlling for Ability: {coef_naive - coef_adjusted:.
↪3f}")

print("\n  Key Insight:")
print("  • Naive regression overestimates the effect")
print("  • This is because Ability (confounder) is correlated with both")
print("  • Controlling for Ability removes the bias")
print("  • The DAG told us we MUST control for Ability!")
```

```
Regression Results:

TRUE causal effect of Education on Income: 2.0

Naive estimate (biased):    3.226
Adjusted estimate (correct): 1.969

Bias from not controlling for Ability: 1.257

  Key Insight:
  • Naive regression overestimates the effect
```

- This is because Ability (confounder) is correlated with both
- Controlling for Ability removes the bias
- The DAG told us we MUST control for Ability!

1.7.3 Visualizing the Confounding

[14]:
```
# Visualize the relationship
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: Naive relationship (confounded)
axes[0].scatter(df['education'], df['income'], alpha=0.5, s=30)
x_range = np.array([df['education'].min(), df['education'].max()])
axes[0].plot(x_range, model_naive.predict(x_range.reshape(-1, 1)),
             'r-', linewidth=2, label=f'Naive slope: {coef_naive:.2f}')
axes[0].set_xlabel('Education (years)')
axes[0].set_ylabel('Income ($1000s)')
axes[0].set_title('Naive Relationship (Confounded)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Right: Color by ability (showing confounding)
scatter = axes[1].scatter(df['education'], df['income'],
                          c=df['ability'], cmap='viridis',
                          alpha=0.6, s=30)
axes[1].plot(x_range, 30 + 2 * x_range,  # True relationship
             'r--', linewidth=2, label=f'True causal slope: 2.0')
axes[1].set_xlabel('Education (years)')
axes[1].set_ylabel('Income ($1000s)')
axes[1].set_title('Relationship Colored by Ability (Confounder)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)
cbar = plt.colorbar(scatter, ax=axes[1])
cbar.set_label('Ability', rotation=270, labelpad=15)

plt.tight_layout()
plt.show()

print("\nVisualization shows:")
print("  • High ability people (yellow) tend to have both higher education AND␣
 ↪higher income")
print("  • Low ability people (purple) tend to have both lower education AND␣
 ↪lower income")
print("  • This confounding makes the naive slope steeper than the true causal␣
 ↪effect")
```



Visualization shows:

- High ability people (yellow) tend to have both higher education AND higher income
- Low ability people (purple) tend to have both lower education AND lower income
- This confounding makes the naive slope steeper than the true causal effect

---

1.8 Part 5: The Collider Bias Example

Let's demonstrate why controlling for a collider creates bias.

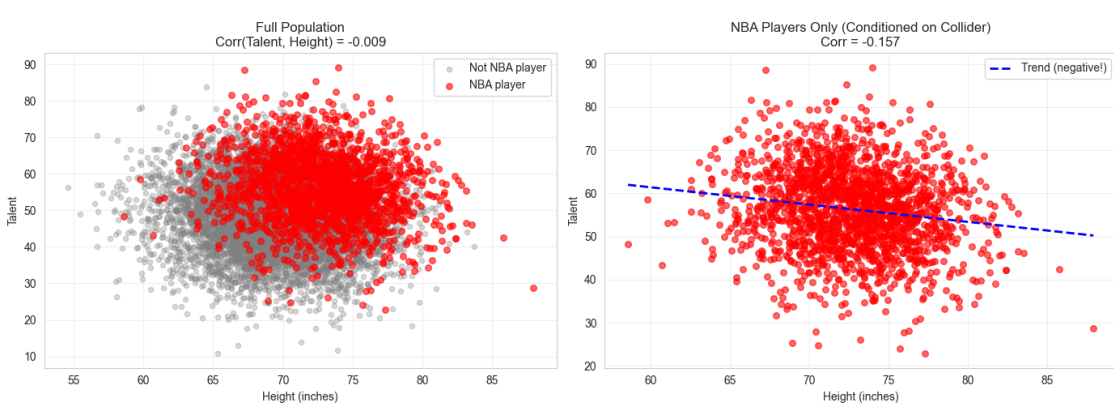
[15]:
```
# Simulate collider bias example: Talent and Height both affect NBA selection
def simulate_nba_collider(n: int = 10000) -> pd.DataFrame:
    """
    Simulate: Talent → NBA_Player ← Height

    Talent and Height are INDEPENDENT in population,
    but become NEGATIVELY correlated if we condition on NBA_Player.
    """
    np.random.seed(42)

    # Talent and Height are independent
    talent = np.random.normal(50, 10, n)
    height = np.random.normal(70, 4, n)  # inches

    # Probability of being NBA player depends on BOTH
    # Standardize for logistic function
    talent_std = (talent - talent.mean()) / talent.std()
    height_std = (height - height.mean()) / height.std()

    # Higher talent OR higher height → higher probability
```

```
    prob_nba = 1 / (1 + np.exp(-(talent_std + height_std - 2)))  # Threshold at␣
↪2
    nba_player = np.random.binomial(1, prob_nba)

    return pd.DataFrame({
        'talent': talent,
        'height': height,
        'nba_player': nba_player
    })

# Generate data
df_nba = simulate_nba_collider(n=10000)

# Correlation in full population
corr_full = df_nba['talent'].corr(df_nba['height'])

# Correlation among NBA players only (conditioning on collider)
df_nba_only = df_nba[df_nba['nba_player'] == 1]
corr_nba = df_nba_only['talent'].corr(df_nba_only['height'])

print("Collider Bias Demonstration:\n")
print(f"Correlation (Talent, Height) in full population: {corr_full:.3f}")
print(f"  → Nearly zero, as expected (they're independent!)")

print(f"\nCorrelation (Talent, Height) among NBA players: {corr_nba:.3f}")
print(f"  → NEGATIVE! Conditioning on collider creates spurious association")

print(f"\nNumber of NBA players: {df_nba_only.shape[0]} out of {len(df_nba)}")
print(f"Percentage: {100 * df_nba_only.shape[0] / len(df_nba):.1f}%")
```

```
Collider Bias Demonstration:

Correlation (Talent, Height) in full population: -0.009
  → Nearly zero, as expected (they're independent!)

Correlation (Talent, Height) among NBA players: -0.157
  → NEGATIVE! Conditioning on collider creates spurious association

Number of NBA players: 1797 out of 10000
Percentage: 18.0%
```

[16]:
```
# Visualize collider bias
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: Full population
axes[0].scatter(df_nba[df_nba['nba_player'] == 0]['height'],
                df_nba[df_nba['nba_player'] == 0]['talent'],
```

```
                alpha=0.3, s=20, label='Not NBA player', color='gray')
axes[0].scatter(df_nba_only['height'], df_nba_only['talent'],
                alpha=0.6, s=30, label='NBA player', color='red')
axes[0].set_xlabel('Height (inches)')
axes[0].set_ylabel('Talent')
axes[0].set_title(f'Full Population\nCorr(Talent, Height) = {corr_full:.3f}')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Right: NBA players only
axes[1].scatter(df_nba_only['height'], df_nba_only['talent'],
                alpha=0.6, s=30, color='red')
# Add trend line
z = np.polyfit(df_nba_only['height'], df_nba_only['talent'], 1)
p = np.poly1d(z)
axes[1].plot(df_nba_only['height'].sort_values(),
             p(df_nba_only['height'].sort_values()),
             'b--', linewidth=2, label=f'Trend (negative!)')
axes[1].set_xlabel('Height (inches)')
axes[1].set_ylabel('Talent')
axes[1].set_title(f'NBA Players Only (Conditioned on Collider)\nCorr =␣
 ↪{corr_nba:.3f}')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n Key Insight (Collider Bias):")
print("  • Among NBA players, shorter players tend to be more talented")
print("  • This is NOT because height causes talent (or vice versa)")
print("  • It's because to make the NBA, short players need exceptional talent")
print("  • Tall players can make it with less talent")
print("  • Conditioning on NBA_Player (collider) creates spurious negative␣
 ↪correlation!")
```



```
Key Insight (Collider Bias):
  • Among NBA players, shorter players tend to be more talented
  • This is NOT because height causes talent (or vice versa)
  • It's because to make the NBA, short players need exceptional talent
  • Tall players can make it with less talent
  • Conditioning on NBA_Player (collider) creates spurious negative correlation!
```

1.9 Part 6: Practical DAG Analysis

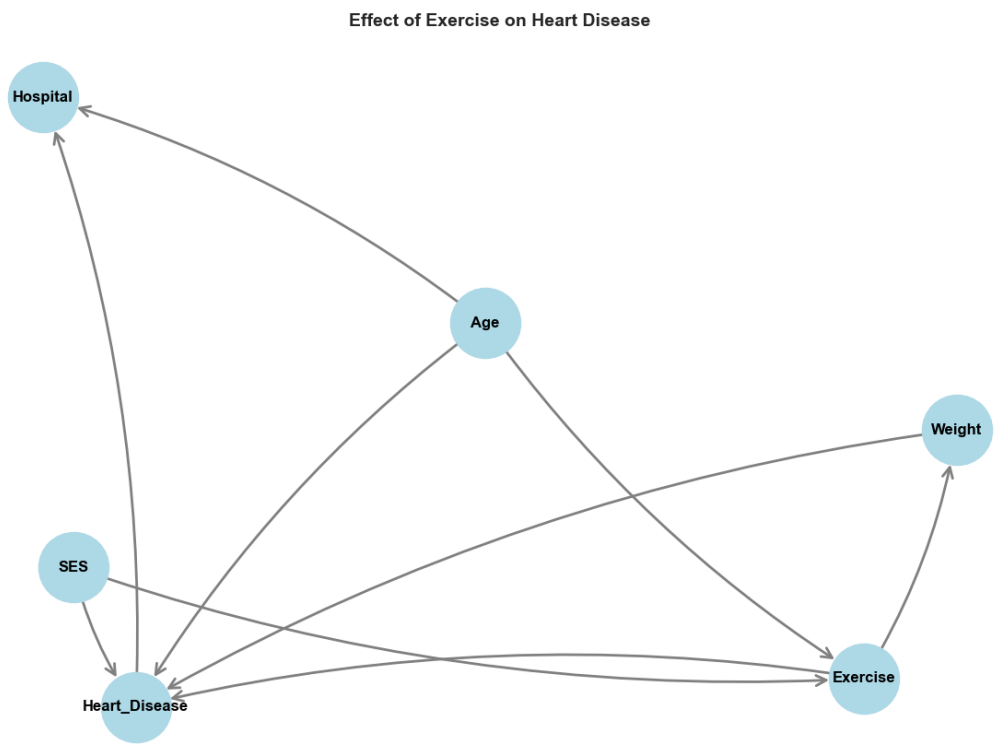
Let's put it all together with a realistic example.

1.9.1 Example: Effect of Exercise on Heart Disease

[17]:
```
# Realistic DAG: Exercise → Heart Disease
edges_health = [
    # Confounders
    ('Age', 'Exercise'),
    ('Age', 'Heart_Disease'),
    ('SES', 'Exercise'),  # Socioeconomic status
    ('SES', 'Heart_Disease'),
    # Direct effect
    ('Exercise', 'Heart_Disease'),
    # Mediator
    ('Exercise', 'Weight'),
    ('Weight', 'Heart_Disease'),
    # Collider
    ('Heart_Disease', 'Hospital'),
    ('Age', 'Hospital')
]

G_health = draw_dag(edges_health,
                    title="Effect of Exercise on Heart Disease",
                    figsize=(12, 8))

print("\nDAG Analysis: Exercise → Heart_Disease\n")

print("CONFOUNDERS (must control):")
print("  • Age: Older people exercise less AND have more heart disease")
print("  • SES: Wealthy people exercise more AND have better health")

print("\nMEDIATOR (must NOT control if we want total effect):")
print("  • Weight: Exercise reduces weight, which reduces heart disease")
print("  • Controlling for Weight blocks part of the causal pathway")
```

```
print("\nCOLLIDER (must NOT control):")
print("  • Hospital: Heart disease and age both increase hospitalization")
print("  • Conditioning on Hospital (e.g., studying only hospitalized␣
↪patients)")
print("    creates selection bias")

print("\n  Correct adjustment set: {Age, SES}")
print("  Wrong: controlling for Weight or Hospital")
```

```
/var/folders/56/jqj8nyt12sxbm6rgrnlf06y00000gn/T/ipykernel_7987/3657559720.py:45
: UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```




```
DAG Analysis: Exercise → Heart_Disease

CONFOUNDERS (must control):
  • Age: Older people exercise less AND have more heart disease
  • SES: Wealthy people exercise more AND have better health

MEDIATOR (must NOT control if we want total effect):
```

```
  • Weight: Exercise reduces weight, which reduces heart disease
  • Controlling for Weight blocks part of the causal pathway

COLLIDER (must NOT control):
  • Hospital: Heart disease and age both increase hospitalization
  • Conditioning on Hospital (e.g., studying only hospitalized patients)
    creates selection bias

 Correct adjustment set: {Age, SES}
 Wrong: controlling for Weight or Hospital
```

1.10 Summary and Key Takeaways

1.10.1 Core Concepts

1. **DAGs represent causal assumptions**:
   - Nodes = variables
   - Arrows = direct causal effects
   - No cycles (acyclic)
   - Based on domain knowledge, NOT learned from data
2. **Three fundamental structures**:
   - **Chain** ($X \to M \to Y$): M mediates; don't control for total effect
   - **Fork** ($X \leftarrow Z \to Y$): Z confounds; DO control to block backdoor
   - **Collider** ($X \to C \leftarrow Y$): C is common effect; DON'T control (creates bias)
3. **Paths and confounding**:
   - **Causal paths**: Follow arrow directions
   - **Backdoor paths**: Confounding paths (go against arrows)
   - Must block all backdoor paths to estimate causal effects
4. **Control strategy**:
   - Control for confounders (common causes in forks)
   - Don't control for mediators (blocks causal path)
   - Don't control for colliders (creates selection bias)
5. **DAG + Data**:
   - DAG tells us WHAT to control for
   - Data tells us HOW MUCH the effect is
   - Without the right DAG, regression can give biased estimates

1.10.2 Why DAGs Matter

- **Make assumptions explicit**: Forces you to think through causal structure
- **Identify confounders**: Shows which variables to control for
- **Avoid bias**: Prevents controlling for mediators or colliders
- **Communicate**: Visual representation everyone can understand
- **Connect to potential outcomes**: Different framework, same insights

1.10.3 DAGs vs Potential Outcomes

Potential Outcomes	DAGs
Mathematical/algebraic	Visual/graphical
Defines causal effects precisely	Shows causal structure
Good for: formal proofs	Good for: intuition, communication
Asks: What's the effect?	Asks: What do we control for?

They're **complementary**: use both!

1.11 Exercises

Try these to reinforce your understanding:

1. **Draw a DAG**: For a research question in your field, draw a DAG showing treatment, outcome, and at least 3 other variables. Identify confounders, mediators, and potential colliders.
2. **Simpson's Paradox**: Create a simulation where controlling for a confounder reverses the direction of an association (positive → negative or vice versa).
3. **M-bias**: Research the "M-bias" or "butterfly bias" structure. Create a DAG and explain when controlling for a variable can introduce bias even if it's not on the causal path.
4. **Multiple paths**: Create a DAG with at least 3 different paths between treatment and outcome. Use NetworkX to find all paths programmatically.

1.12 Further Reading

- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. Chapter 1-3.
- Pearl, J., Glymour, M., & Jewell, N. P. (2016). *Causal Inference in Statistics: A Primer*. Chapter 2-3.
- Hernán, M. A., & Robins, J. M. (2020). *Causal Inference: What If*. Chapter 6-7.
- Elwert, F. (2013). "Graphical Causal Models." *Handbook of Causal Analysis for Social Research*.

Interactive Tools: - DAGitty: http://www.dagitty.net/ (draw and analyze DAGs online) - DoWhy library: Microsoft's Python library for causal inference with DAGs

---

Next: Confounding Examples - Deep dive into identifying and handling confounders